



fahre fort

Now I have a thorough understanding of the model structure. Let me provide a comprehensive answer on runtime optimization techniques.

Runtime Optimization Techniques for Your MIQCP Model

Based on my analysis of your `hdv_disposition_optimization.py` (which is actually a **MILP** — the quadratic degradation term was already linearized via tangent approximation), here are concrete techniques to improve solve time without changing functionality:

1. Constraint & Variable Reduction (Highest Impact)

Technique	Where in Your Model	Why It Helps
Tighten <code>event_possible_starts / possible_start_times</code>	<code>z_m_f_s</code> domain (line 1974–1978)	Fewer (m, f, s) tuples $\rightarrow$ exponentially fewer binary variables. Pre-filter start times that violate depot return deadlines, break rules, or charger availability <i>before</i> model building.
Aggregate time steps where possible	<code>time_steps</code> ($48 \times 30\text{-min}$)	If price/load curves are flat over 2–4 steps, merge them. Reduces <code>E_pos</code> , <code>E_neg</code> , <code>x_m_SoC</code> , <code>z_m_f_s</code> dimensionality by 2–4 $\times$.
Eliminate <code>x_m_t_l</code> for internal locations	Lines 1979, 1995–2005	You already removed <code>x_m_t_l_bev</code> (good). Keep <code>x_m_t_l</code> only for <code>external_charging</code> ; drop depot/internal location binaries entirely — they’re implied by <code>z_chg / v2g_z_t</code> .
Pre-compute feasible (m, f) pairs	<code>z_m_f_s</code> sum (line 1992–1993)	Skip vehicles that can’t reach trip <code>f</code> (range, time window, depot return). Build <code>vehicle_trip_candidates[m] = [f...]</code> and only create <code>z_m_f_s</code> for those.
Reduce chaining_on binary variables	<code>route_start</code> , <code>route_end</code> , <code>chain</code> (lines 2047–2057)	Only create chain variables for trip pairs that are <i>actually</i> chainable (spatial + temporal overlap). Your <code>link_steps</code> already does this — ensure it’s aggressively pruned.

2. Strengthen LP Relaxation (Big-M & Formulation)

Issue	Current Code	Fix
Loose big-M in full_binds	Lines 2655–2663: <code>m_station = best_station_e, m_battery = battery_factor * cap</code>	Already per-constraint (good). Further tighten: <code>m_station = min(best_station_e, battery_factor * headroom_max)</code> where <code>headroom_max = cap * (1 - initial_soc_fraction)</code> .
E_private pinning uses <code>==</code> with <code>min()</code>	Line 2644 ($t=0$) and 2658–2663 ($t>0$)	For $t>0$, the binary <code>full_binds + big-M</code> is correct. Consider adding: <code>E_private <= best_station_e * z_chg</code> and <code>E_private <= battery_factor * headroom</code> (already implied but helps LP bound).
site_peak_kW uses <code>max_</code> over steps	Not shown but likely <code>site_peak_kW = max_(t) site_import[t]</code>	Replace with epigraph: <code>site_peak_kW >= site_import[t] ∀t</code> + minimize <code>site_peak_kW</code> . Avoids non-linear <code>max_</code> if present.
Symmetry in vehicle indices	Vehicles of same type are symmetric	Add symmetry-breaking: <code>y_m >= y_{m+1}</code> for identical vehicles, or fix first vehicle of each type to be used if any is used.

3. Gurobi Parameter Tuning (Immediate Wins)

Your current `solve_model` (lines 3356–3384) uses conservative defaults. For this time-indexed scheduling MILP, try:

```
def solve_model(model, optimization_MIPGap, gurobi_threads):
    model.setParam('OutputFlag', 1)          # Enable log to see progress
    model.setParam('LogToConsole', 1)
    model.setParam('MIPGap', optimization_MIPGap)
    model.setParam('Threads', gurobi_threads)
    model.setParam('TimeLimit', 3600)        # 1h wall-clock cap (adjust)

    # --- Key changes for scheduling MILPs ---
    model.setParam('MIPFocus', 1)            # 1 = feasibility, 2 = optimality, 3 = both
    model.setParam('Cuts', 2)                # Aggressive cuts (Gomory, MIR, flow cuts)
    model.setParam('CutPasses', 20)          # More cut rounds at root
    model.setParam('Heuristics', 0.5)        # More primal heuristics (RINS, local branching)
    model.setParam('RINS', 25)               # RINS heuristic every 25 nodes
    model.setParam('Polishing', 1)           # Solution polishing
    model.setParam('Symmetry', 2)            # Aggressive symmetry breaking
    model.setParam('Presolve', 2)            # Aggressive presolve
    model.setParam('AggFill', 10)            # Allow more fill-in during aggregation
    model.setParam('PreDual', 1)             # Dual reductions in presolve
    model.setParam('PreMIQCPForm', 1)        # Reformulate QCPs (if any remain)

    # For degenerate time-indexed LPs:
    model.setParam('DegenMoves', 0)          # Your current setting (good)
    model.setParam('Method', 2)              # Barrier for root LP (often faster)
    model.setParam('Crossover', 1)           # Crossover to basic solution
    model.setParam('BarConvTol', 1e-4)       # Looser barrier convergence

    # Node selection:
```

```

model.setParam('NodeMethod', 1)           # Dual simplex at nodes
model.setParam('VarBranch', 1)           # Strong branching

model.optimize()

```

Run `model.tune()` **once** (lines 3387–3390 uncommented) on a representative instance — it will search parameter space and often finds 2–5× speedups.

4. Decomposition & Algorithmic Strategies

Strategy	Applicability	Implementation Sketch
Benders / Logic-based Benders	If y_m (vehicle use) decisions are hard	Master: select vehicles & routes. Subproblem: fix y_m , optimize energy flows (LP). Add feasibility/optimality cuts.
Column Generation (Branch-and-Price)	If many trips, few vehicles	Price routes (vehicle-day schedules) instead of assigning trips individually. Your <code>route_start/chain</code> structure is already close.
Fix-and-Dive / Local Branching	Good feasible solutions exist	Solve relaxed LP → fix 80% of z_m, f_s to rounded values → solve MIP on remainder. Iterate.
Rolling Horizon	Multi-day design model	Solve day-by-day with overlap (e.g., 36h windows, fix first 24h). Reduces problem size per solve.
Warm-start from heuristic	You have <code>run_scheduler</code>	Run a fast greedy scheduler first, feed z_m, f_s, x_m, t_E, y_m as <code>Start</code> values (<code>var.Start = val</code>).

5. Model-Specific Reformulations

a) V2G / Charging Virtual Trips (Lines 1973, 2563–2609)

Instead of binary `v2g_z_t` / `chg_z_t` per vehicle per step, use **SOS1** or **indicator constraints**:

```

# Current: binary v2g_z_t + big-M on E_neg
# Better: model.addGenConstrIndicator(v2g_z_t, True, E_neg >= min_discharge)
#         model.addGenConstrIndicator(v2g_z_t, False, E_neg == 0)
# Gurobi handles this natively with stronger LP relaxation.

```

b) E_{private} / E_{public} Split (Lines 2614–2664)

The `min(best_station_e, battery_factor * headroom)` with binary `full_binds` is correct but adds 1 binary × BEV × timestep. Alternative:

- If `best_station_e` is constant per vehicle (strongest station), pre-compute which side binds at each SoC breakpoint. Use **piecewise-linear (PWL)** approximation of the min function — Gurobi's `addGenConstrPWL` creates fewer binaries with logarithmic encoding.

c) Degradation Cost Linearization (Lines 2935–2940)

You already use tangent linearization (good). Ensure `soc_weight_tangents` has **minimal breakpoints** (4–5 is usually enough for quadratic). More tangents = more constraints = slower LP solves.

d) Route Chaining Flow Balance (Lines 2089–2110)

The `route_start/route_end/chain` flow constraints create a dense submatrix. Consider **network flow formulation** with single commodity flow on a trip graph — reduces binary count from $O(|trips|^2)$ to $O(|trips|)$.

6. Practical Immediate Checklist

1. **Profile the model build vs. solve time** — add timers around `model_build` and `model.optimize()`. If build > 30% of total, optimize data structures (pre-compute dicts, avoid `gp.quicksum` in loops).
2. **Write .lp or .mps and inspect** (`model.write('model.lp')`) — look for:
 - Rows/columns count
 - Density of constraint matrix
 - Big-M coefficients (should be as small as possible)
3. **Run with MIPFocus=1 first** — if feasible solution found quickly but bound stalls, switch to MIPFocus=2 or 3.
4. **Enable LogFile** and analyze root relaxation gap, cut effectiveness, heuristic hits.
5. **Test `model.tune()`** on a small instance (10 vehicles, 20 trips) for 30 min — apply best parameters to full runs.

7. Expected Impact Ranking

Technique	Est. Speedup	Effort
Aggressive <code>event_possible_starts</code> pruning	2–10×	Low
Gurobi parameter tune (MIPFocus, Cuts, Heuristics, RINS)	1.5–5×	Very Low
Symmetry breaking on identical vehicles	1.2–3×	Low
Rolling horizon / fix-and-divide heuristic	3–20×	Medium
Column generation (route-based)	5–50×	High
Benders decomposition	2–10×	High

Recommendation: Start Here (This Week)

1. Add a **pre-processing filter** that removes (m, f, s) tuples where vehicle m cannot physically complete trip f starting at s (range, depot return, driver hours).
2. Run `model.tune()` on a representative instance — commit the best parameter file.
3. Switch `MIPFocus=1 + Heuristics=0.5 + RINS=25` in `solve_model` immediately.
4. Add **warm-start** from your greedy scheduler (`run_scheduler`) by setting `var.Start` on `z_m_f_s, y_m, x_m_t_E`.

These four changes alone typically yield 3–10× **speedup** on fleet scheduling MILPs of this structure, with zero functional changes.

Would you like me to generate a patch file with the parameter changes and warm-start integration, or dive deeper into any specific technique (e.g., Benders decomposition sketch for your model)?